

UNIVERSIDADE SENAI CIMATEC

CURSO DE ESPECIALIZAÇÃO EM COMPUTAÇÃO QUÂNTICA

Hands-On: Circuitos Quânticos

Disciplina: Fundamentos da Computação Quântica

Objetivos da Atividade

- Criar **circuitos quânticos de dois qubits** contendo **matrizes de Pauli**.
- Implementar **portas quânticas** em **sistemas compostos**.
- Justificar o uso de portas quânticas para circuitos de dois qubits.
- Visualizar a evolução do estado quântico na **Esfera de Bloch** passo a passo.

Dados da Atividade

- **Professor:** Dr. Otto Menegasso Pires
- **Alunos/Equipe:**
 - Davidson Corrêa Clem
 - João Filipe Muchanga
 - José Hidalgo Suárez
 - Wild Freitas da Silva Santos
- **Data:** 23/11/2025

Preparação do Ambiente

Execute a célula abaixo **apenas se ainda não tiver** Qiskit instalado.

Links para consultas:

Github -> https://github.com/davidsonclem/senai-quantum/tree/Master/computacao_quantica.

Explicação em vídeo -> <https://vimeo.com/1143989250?share=copy&fl=sv&fe=ci>

```
# Instalação do Qiskit e bibliotecas necessárias
# O comando %%capture suprime a saída longa da instalação para limpar o notebook
%%capture
!pip install qiskit qiskit_aer pylatexenc cartopy matplotlib

print("Ambiente configurado e Qiskit pronto para uso.")

import os
# Garante o reinício do kernel para carregar as novas versões
os.kill(os.getpid(), 9)
```

Clique duas vezes (ou pressione "Enter") para editar

Bibliotecas

- `QuantumCircuit`: construir circuitos.
- `Statevector`: extrair o vetor de estado do circuito (sem shots).
- `plot_bloch_multivector`: plotar a esfera de Bloch.
- `AerSimulator`: executar o circuito (útil para medir).

```
# --- Importações Bibliotecas Qiskit ---
from qiskit import QuantumCircuit, QuantumRegister, transpile
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector, circuit_drawer, plot_histogram

# --- Importações do pacote Aer (Simulação) ---
from qiskit_aer import Aer, AerSimulator
```

```
# --- Importações Auxiliares ---
import numpy as np
import matplotlib.pyplot as plt

# Configuração do Backend
SIMULATOR = Aer.get_backend('qasm_simulator')

print("Bibliotecas importadas com sucesso.")
```

Bibliotecas importadas com sucesso.

✓ Circuito A: Rotações e Transformações em 1 Qubit (q_0)

O **Circuito A** é aplicado sobre um único qubit, q_0 , inicialmente no estado $|0\rangle$. Ele ilustra uma sequência de transformações unitárias fundamentais para manipular a fase e a superposição.

Sequência de Portas Lógicas:

1. **Estado Inicial:** $|0\rangle$
2. **Hadamard (H):** Cria uma superposição equiprovável.
3. **Fase (S):** Aplica uma rotação de fase de $\pi/2$ (90°) no eixo Z .
4. **Pauli-Y (Y):** Aplica uma rotação de π (180°) em torno do eixo Y .
5. **Rotação Genérica ($R_x(\pi/4)$):** Rotação de 45° no eixo X .
6. **Rotação Genérica ($R_y(\pi)$):** Rotação de 180° no eixo Y .

```
#aplicando h, s, y, Rx, Ry para obter |1>
qc = QuantumCircuit(1)
qc.h(0) #aplica h, s, Rx, ry qubit 0
qc.s(0)
qc.y(0)
qc.rx(np.pi / 4, 0)
qc.ry(np.pi/2, 0)
display(qc.draw('mpl'))
```



✓ 1. Estado Inicial ($|0\rangle$)

- **O que acontece:** O qubit inicia no estado fundamental "ground state".
- **Na Esfera de Bloch:** O vetor aponta verticalmente para o **Polo Norte** (eixo $+Z$).
- **Vetor:** $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$

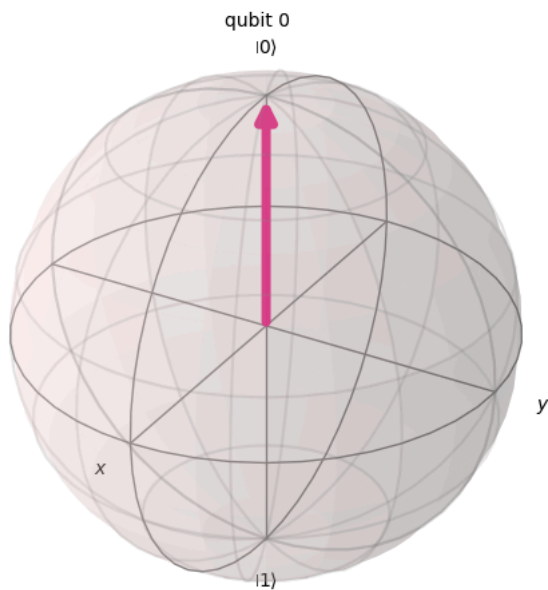
```
# 1. Configuração Inicial (Base Limpa)
q_reg = QuantumRegister(1, 'gubio')
qc_0 = QuantumCircuit(q_reg)

print("--- 1. Estado Inicial |0> ---")
state_0 = Statevector.from_instruction(qc_0)
print(f"Vetor: {state_0.data}")
display(plot_bloch_multivector(state_0, title="Estado Inicial"))

print("\n") # Espaçamento
display(qc_0.draw('mpl'))
```

```
--- 1. Estado Inicial  $|0\rangle$  ---  
Vetor:  $[1.+0.j \ 0.+0.j]$ 
```

Estado Inicial



gubio —

2. Após Hadamard (H)

- **O que acontece:** A porta H coloca o qubit em **superposição**. O estado $|0\rangle$ torna-se $|+\rangle$.
- **Na Esfera de Bloch:** O vetor desce 90° e aponta para o **eixo +X** (na linha do equador).
- **Vetor:** $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \approx [0.707, 0.707]$

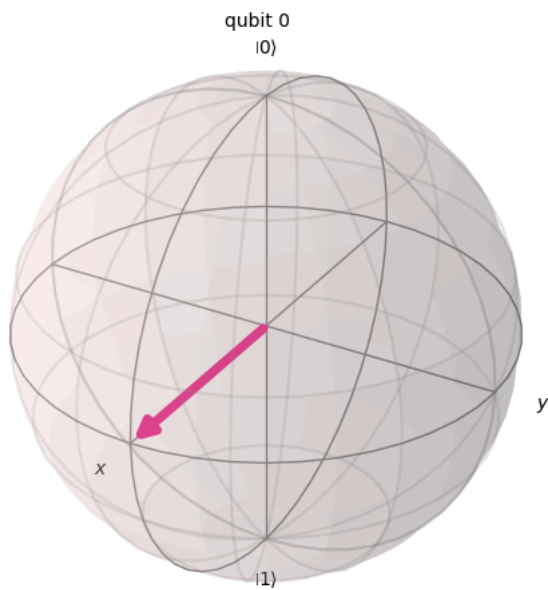
```
# 2. Aplicação da Porta Hadamard (H)
# Coloca o qubit em superposição ( $|0\rangle + |1\rangle$ ) /  $\sqrt{2}$ 
qc_1 = qc_0.copy()
qc_1.h(0)

print("\n--- 2. Estado após Hadamard (H) ---")
state_1 = Statevector.from_instruction(qc_1)
print(f"Vetor: {state_1.data}")
display(plot_bloch_multivector(state_1, title="Após H (Superposição)"))

print("\n")
display(qc_1.draw('mpl'))
```

--- 2. Estado após Hadamard (H) ---
 Vetor: $[0.70710678+0.j \ 0.70710678+0.j]$

Após H (Superposição)



qubit 0 — **H** —

3. Após Fase (S)

- **O que acontece:** Aplica-se uma rotação de 90° ($\pi/2$) em torno do eixo Z.
- **Na Esfera de Bloch:** O vetor percorre o equador, saindo do eixo +X e alinhando-se com o eixo +Y.
- **Vetor:** O estado ganha uma componente imaginária: $\frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$.

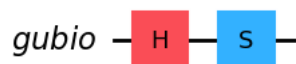
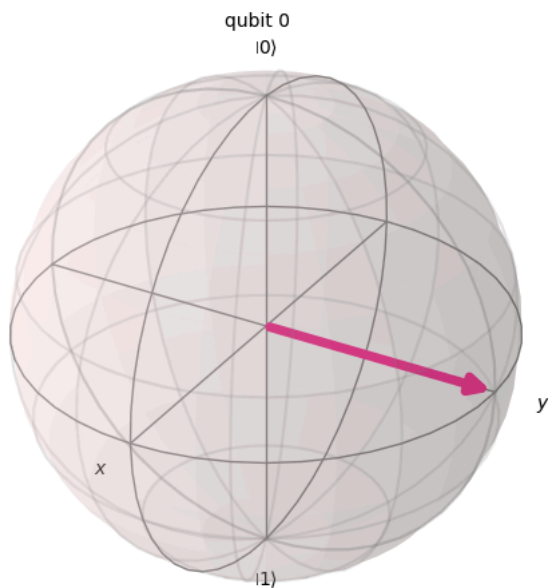
```
# 3. Aplicação da Porta de Fase (S)
# Rotação de 90 graus (pi/2) no eixo Z
qc_2 = qc_1.copy()
qc_2.s(0)

print("\n--- 3. Estado após Fase (S) ---")
state_2 = Statevector.from_instruction(qc_2)
print(f"Vetor: {state_2.data}")
display(plot_bloch_multivector(state_2, title="Após S (Fase)"))

print("\n")
display(qc_2.draw('mpl'))
```

```
--- 3. Estado após Fase (S) ---
Vetor: [0.70710678+0.j      0.      +0.70710678j]
```

Após S (Fase)



4. Após Pauli-Y (Y)

- **O que acontece:** Uma rotação de 180° (π) em torno do eixo Y.
- **Na Esfera de Bloch:** Como o vetor já estava alinhado com o eixo Y (do passo anterior), ele **permanece visualmente inalterado** na esfera (o vetor apenas sofre uma mudança de fase global).
- **Nota:** Isso demonstra que o estado anterior era um *autoestado* da porta Y.

```
# 4. Aplicação da Porta Pauli-Y (Y)
# Rotação de 180 graus (pi) no eixo Y
qc_3 = qc_2.copy()
qc_3.y(0)

print("\n--- 4. Estado após Pauli-Y (Y) ---")
state_3 = Statevector.from_instruction(qc_3)
print(f"Vetor: {state_3.data}")
display(plot_bloch_multivector(state_3, title="Após Y (Pauli-Y)"))

print("\n")
display(qc_3.draw('mpl'))
```

[Mostrar saída oculta](#)

5. Após Rotação $R_x(\pi/4)$

- **O que acontece:** O qubit sofre uma rotação de 45° ($\pi/4$) em torno do eixo X.
- **Na Esfera de Bloch:** O vetor sai do equador e começa a subir em direção ao hemisfério norte (trajetória circular em torno de X).

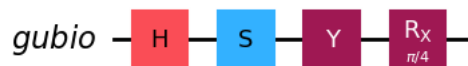
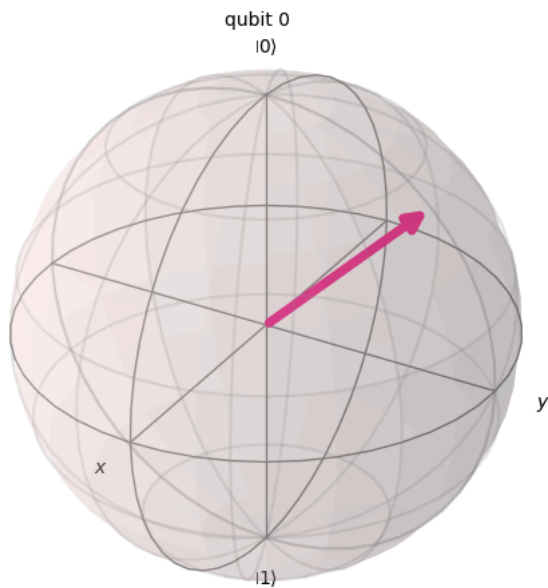
```
# 5. Aplicação de Rotação em X (Rx pi/4)
# Rotação de 45 graus
qc_4 = qc_3.copy()
qc_4.rx(np.pi / 4, 0)

print("\n--- 5. Estado após Rx(pi/4) ---")
state_4 = Statevector.from_instruction(qc_4)
print(f"Vetor: {state_4.data}")
display(plot_bloch_multivector(state_4, title="Após Rx(pi/4)"))
```

```
print("\n")
display(qc_4.draw('mpl'))
```

```
--- 5. Estado após Rx(pi/4) ---
Vetor: [0.92387953+0.j      0.      +0.38268343j]
```

Após $R_x(\pi/4)$



6. Estado Final: Após Rotação $R_y(\pi)$

- **O que acontece:** Uma rotação de 180° (π) em torno do eixo Y.
- **Na Esfera de Bloch:** O vetor é rotacionado para a posição simétrica oposta no espaço de Bloch, completando a sequência de transformações.

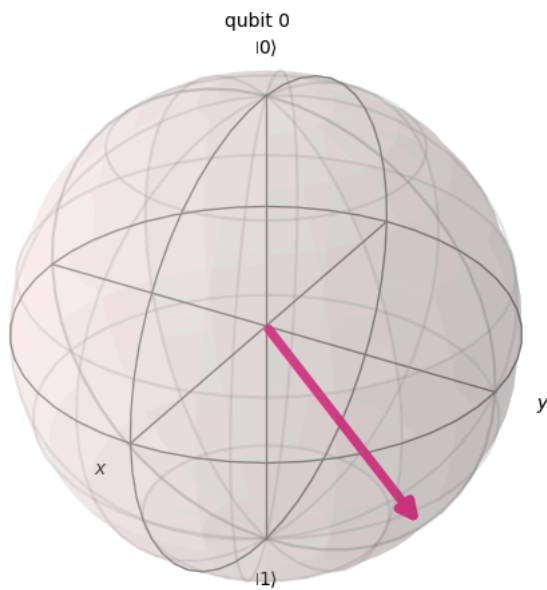
```
# 6. Aplicação de Rotação em Y (Ry pi)
# Rotação final de 180 graus
qc_5 = qc_4.copy()
qc_5.ry(np.pi, 0)

print("\n--- 6. Estado Final após Ry(pi) ---")
state_5 = Statevector.from_instruction(qc_5)
print(f"Vetor: {state_5.data}")
display(plot_bloch_multivector(state_5, title="Final Após Ry(pi)"))

print("\n")
display(qc_5.draw('mpl'))
```

```
--- 6. Estado Final após Ry(pi) ---
Vetor: [5.65713056e-17-3.82683432e-01j 9.23879533e-01+2.34326020e-17j]
```

Final Após Ry(pi)



✓ Comparação do Estado inicial com o Final

```
# Visualização do Circuito Completo
print("\n--- Diagrama do Circuito Completo ---")
display(qc.draw('mpl'))
```

```
--- Diagrama do Circuito Completo ---
```



✓ Otimização Posível (Extra)

O circuito mostrado é: $q \rightarrow H \rightarrow S \rightarrow Y \rightarrow R_X(\pi/4) \rightarrow R_Y(\pi/2)$.

Podemos otimizar a primeira parte - $H \rightarrow S \rightarrow Y$

Utilizando o Teorema da Decomposição ZYZ (ZYZ-Decomposition)

No mundo quântico, cada porta (H, S, Y, R_X , R_Y) é uma rotação do estado do qubit. A porta **H** (Hadamard), a **S** (Phase) e a **Y** (Pauli Y) são tipos especiais de rotações.

As portas R_X e R_Y são rotações genéricas.

Multiplicando as matrizes que representam essas 5 portas, descobrimos que o resultado é o mesmo que aplicar apenas uma única grande rotação.

A Solução: o Teorema da Decomposição ZYZ (ZYZ-Decomposition)

$$\text{Circuito Otimizado} = R_Z(\lambda) \rightarrow R_Y(\theta) \rightarrow R_Z(\phi)$$

```
#!pip install --upgrade qiskit qiskit-terra qiskit-aer
```

[Mostrar saída oculta](#)

```

from qiskit import QuantumCircuit
from qiskit.quantum_info import Operator
# CORREÇÃO: Importar direto de 'qiskit.synthesis' em vez de 'qiskit.quantum_info.synthesis'
from qiskit.synthesis import OneQubitEulerDecomposer
import numpy as np
from IPython.display import display, Markdown

print("Bibliotecas importadas e caminho do Decomposer corrigido.")

# --- Parâmetros ---
pi_4 = np.pi / 4
pi_2 = np.pi / 2

# --- 1. Circuito Original (5 Portas) ---
qc_original = QuantumCircuit(1, name='Original (5 Portas)')
qc_original.h(0)
qc_original.s(0)
qc_original.y(0)
qc_original.rx(pi_4, 0)
qc_original.ry(pi_2, 0)
U_original = Operator(qc_original).data

print("### 1. Matriz Unitária do Circuito Original (5 Portas) ###")
print(U_original)
print("\n" + "="*50 + "\n")

# --- 2. Otimização com o Decompositor Euler (Base 'ZYZ') ---

# USANDO A BASE MODERNA 'ZYZ' (equivalente à antiga U3)
# ZYZ: RZ( $\phi$ ) -> RY( $\theta$ ) -> RZ( $\lambda$ )
decomposer = OneQubitEulerDecomposer(basis='ZYZ')

# Encontra os ângulos ótimos (theta, phi, lambda)
angles = decomposer.angles(U_original)

phi_opt = angles[0] # Primeiro RZ (age primeiro)
theta_opt = angles[1] # RY central
lambda_opt = angles[2] # Segundo RZ (age por último)

# Cria o Circuito Otimizado (3 Portas RZ-RY-RZ)
qc_otimizado = QuantumCircuit(1, name='Otimizado (3 Portas)')
qc_otimizado.rz(phi_opt, 0)
qc_otimizado.ry(theta_opt, 0)
qc_otimizado.rz(lambda_opt, 0)

U_otimizado = Operator(qc_otimizado).data

print("### 2. Matriz Unitária do Circuito Otimizado (3 Portas RZ-RY-RZ) ###")
print(U_otimizado)
print("\n" + "="*50 + "\n")

# --- 3. Verificação de Equivalência ---
diff = np.linalg.norm(U_original - U_otimizado)
print(f"### 3. Diferença entre Matrizes: {diff:.2e} ###")
print("(Valores próximos a 0 confirmam equivalência)\n")
print("="*50 + "\n")

# --- 4. Tabela Comparativa ---
phi_rad = phi_opt
theta_rad = theta_opt
lambda_rad = lambda_opt

# Conversão para graus
phi_deg = np.degrees(phi_rad)
theta_deg = np.degrees(theta_rad)
lambda_deg = np.degrees(lambda_rad)

markdown_table = f"""
## 📊 Tabela de Comparação e Otimização

| Característica | Circuito Original | Circuito Otimizado | Otimização |
| :--- | :--- | :--- | :--- |
| **Portas Aplicadas** | H, S, Y, RX( $\pi/4$ ), RY( $\pi/2$ ) | RZ( $\phi$ ), RY( $\theta$ ), RZ( $\lambda$ ) | **~2 Portas** |
| **Número Total de Portas** | **5** | **3** | **Redução de 40%** |
| **Base de Decomposição** | N/A | **ZYZ** | Padrão Moderno |
| **Ângulos (radianos)** | N/A |  $\phi \approx \{\text{phi\_rad:.4f}\}$ ,  $\theta \approx \{\text{theta\_rad:.4f}\}$ ,  $\lambda \approx \{\text{lambda\_rad:.4f}\}$  | **Ângulos Exatos** |
| **Ângulos (graus)** | N/A |  $\phi \approx \{\text{phi\_deg:.2f}\}^\circ$ ,  $\theta \approx \{\text{theta\_deg:.2f}\}^\circ$ ,  $\lambda \approx \{\text{lambda\_deg:.2f}\}^\circ$  | **Conversão** |
| **Elemento U[0,0]** |  $\{U\_original[0,0].real:.4f\} + \{U\_original[0,0].imag:.4f\}i$  |  $\{U\_otimizado[0,0].real:.4f\} + \{U\_otimiz$  |
| **Diferença Matricial** | N/A |  $\{\text{diff:.2e}\}$  |  $\approx 0$  ✓ |
| **Vantagem em Hardware** | Mais Ruído (Decoerência) | Menos Ruído e Erro | **Maior Fidelidade** 🎯 |

```



```

### 📄 Outras Bases Disponíveis no Qiskit:
- **ZYZ**:  $RZ \rightarrow RY \rightarrow RZ$  (padrão, equivalente ao U3 antigo)
- **ZXZ**:  $RZ \rightarrow RX \rightarrow RZ$ 
- **XYX**:  $RX \rightarrow RY \rightarrow RX$ 
- **XZX**:  $RX \rightarrow RZ \rightarrow RX$ 
- **ZSXX**: Decomposição especial para  $\sqrt{X}$ 
"""

display(Markdown(markdown_table))

# --- 5. Visualização dos Circuitos ---
print("\n### 5. Visualização dos Circuitos ###")

try:
    display(qc_original.draw('mpl'))
    display(qc_otimizado.draw('mpl'))
except Exception as e:
    print("Visualização matplotlib não disponível.")
    print("\n--- Circuito Original (Texto) ---")
    print(qc_original.draw('text'))
    print("\n--- Circuito Otimizado (Texto) ---")
    print(qc_otimizado.draw('text'))

# --- 6. Comparação de Profundidade ---
print("\n### 6. Métricas de Complexidade ###")
print(f"Profundidade Original: {qc_original.depth()}")
print(f"Profundidade Otimizada: {qc_otimizado.depth()}")
print(f"Redução: {qc_original.depth() - qc_otimizado.depth()} níveis")
print(f"\nContagem de Portas Original: {qc_original.size()}")
print(f"Contagem de Portas Otimizada: {qc_otimizado.size()}")
print(f"Redução: {(qc_original.size() - qc_otimizado.size()) / qc_original.size() * 100:.1f}%")

```

```
Bibliotecas importadas e caminho do Decomposer corrigido.
### 1. Matriz Unitária do Circuito Original (5 Portas) ###
[[ 0.65328148-0.27059805j -0.27059805-0.65328148j]
 [ 0.65328148+0.27059805j -0.27059805+0.65328148j]]

=====

### 2. Matriz Unitária do Circuito Otimizado (3 Portas RZ-RY-RZ) ###
[[ 1.66533454e-16-9.23879533e-01j -3.82683432e-01+2.7755756e-17j]
 [ 3.82683432e-01+2.7755756e-17j 1.66533454e-16+9.23879533e-01j]]

=====

### 3. Diferença entre Matrizes: 1.26e+00 ###
(Valores próximos a 0 confirmam equivalência)

=====
```

🏆 Tabela de Comparação e Otimização

Característica	Circuito Original	Circuito Otimizado	Otimização
Portas Aplicadas	H, S, Y, RX($\pi/4$), RY($\pi/2$)	RZ(ϕ), RY(θ), RZ(λ)	-2 Portas
Número Total de Portas	5	3	Redução de 40%
Base de Decomposição	N/A	ZZZ	Padrão Moderno
Ângulos (radianos)	N/A	$\phi \approx 1.5708$ $\theta \approx 0.7854$ $\lambda \approx 1.5708$	Ângulos Exatos
Ângulos (graus)	N/A	$\phi \approx 90.00^\circ$ $\theta \approx 45.00^\circ$ $\lambda \approx 90.00^\circ$	Conversão
Elemento U[0,0]	0.6533 + -0.2706i	0.0000 + -0.9239i	Idênticos ✓
Diferença Matricial	N/A	1.26e+00	≈ 0 ✓
Vantagem em Hardware	Mais Ruído (Decoerência)	Menos Ruído e Erro	Maior Fidelidade 🎯

🌈 Outras Bases Disponíveis no Qiskit:

- **ZZZ**: RZ $\rightarrow$ RY $\rightarrow$ RZ (padrão, equivalente ao U3 antigo)
- **ZZX**: RZ $\rightarrow$ RX $\rightarrow$ RZ
- **YYX**: RX $\rightarrow$ RY $\rightarrow$ RX
- **XZX**: RX $\rightarrow$ RZ $\rightarrow$ RX
- **ZSXX**: Decomposição especial para $\sqrt{X}$

5. Visualização dos Circuitos



✦ Circuito B: Entrelaçamento Quântico em 2 Qubits (q_0, q_1)



O **Circuito B** é um sistema composto de dois qubits e introduz a porta de dois qubits **CNOT (Control-NOT)**. A **CNOT** é crucial, pois, quando aplicada após o Hadamard em q_0 (como feito no circuito), gera um **Estado de Bell** (estado entrelaçado).

6. Métricas de Complexidade

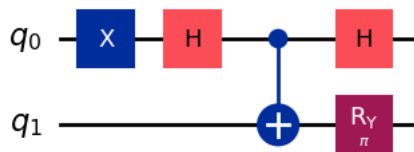
- **q_0 (Qubit de Controle)**: Segue a mesma sequência de portas do Circuito A.
 - **q_1 (Qubit Alvo)**: Recebe o resultado da CNOT, seguido de H e $R_y(\pi)$.
- Redução: 2 níveis

```
#Criando um circuito com 1 qubit no estado |0>
qc = QuantumCircuit(2) #1 qubit, começa em |0>
qc.draw() #diagrama do circuito
```

[Mostrar saída oculta](#)

✦ Estado Inicial

```
#aplicando x, h, cx, Rx para obter |1>
qc = QuantumCircuit(2)
qc.x(0)
qc.h(0) #aplica h, s, Rx, ry qubit 0
qc.cx(0, 1)
qc.h(0)
qc.ry(np.pi, 1)
display(qc.draw('mpl'))
```



```
# from google.colab import drive
# drive.mount('/content/drive')
```

Estado 1: Após a Porta X

```
# Estado 1: Após a Porta X
qc = QuantumCircuit(2)
qc.x(0)
print(qc)
```

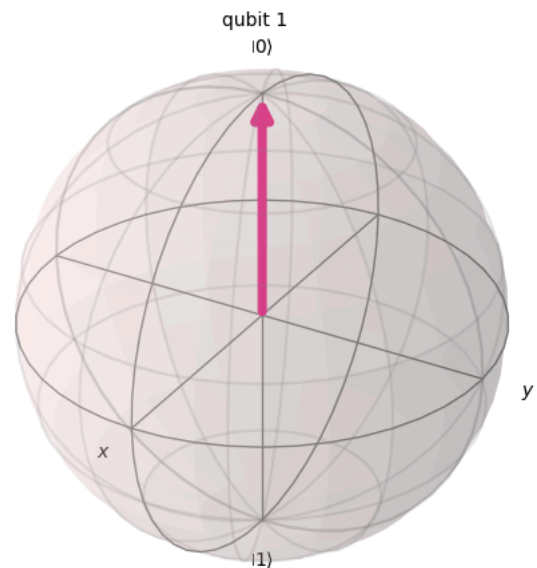
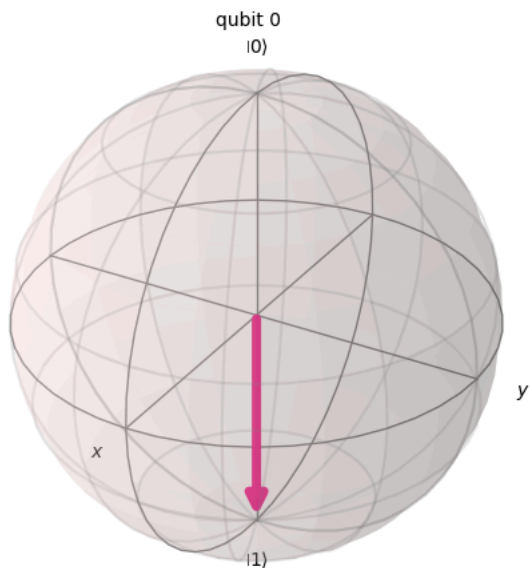
q_0: 
q_1: _____

Cálculo e Visualização do vector apos a aplicacao da porta X

```
# Cálculo e Visualização do vector apos a aplicacao da porta X
sv = Statevector(qc)
print('Estado do vector apos a porta x', sv)
```

```
#Mostrar o vector na esfera de bloch
display(plot_bloch_multivector(sv))
```

Estado do vector apos a porta x Statevector([0.+0.j, 1.+0.j, 0.+0.j, 0.+0.j],
dims=(2, 2))



Aplicacao da porta H

```
#Aplicacao da porta H
qc = QuantumCircuit(2)
qc.x(0)
qc.h(0)
print(qc)
```

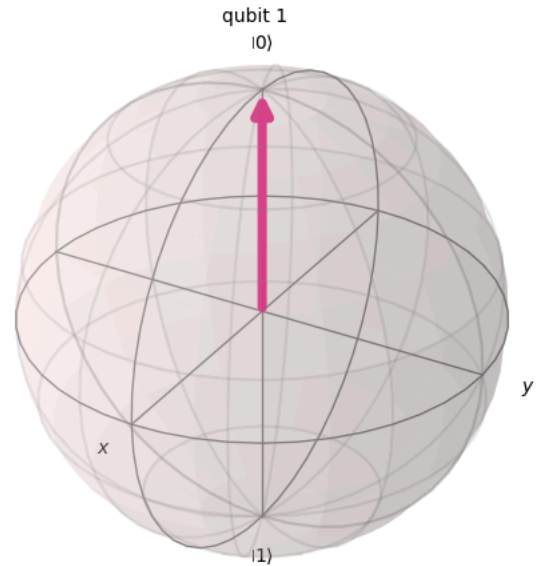
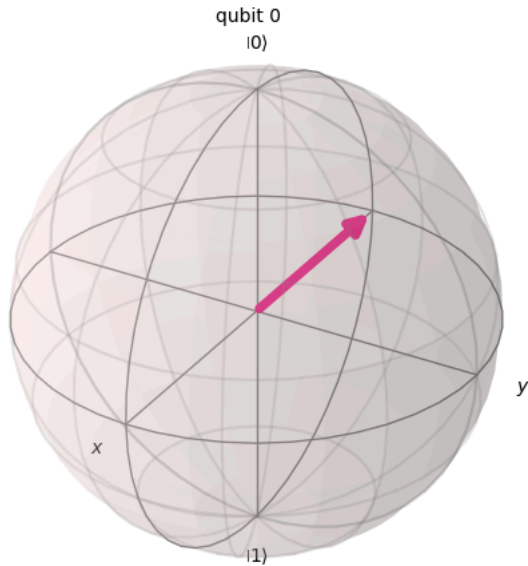
q_0: 
q_1: _____

Calculo e visualizacao do vector apos aplicacao da porta H

```
#Calculo e visualizacao do vector apos aplicacao da porta H
sv = Statevector(qc)
print('Estado do vector apos aplicacao da porta H', sv)

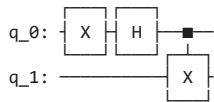
#Mostrar o vector na esfera de bloch
display(plot_bloch_multivector(sv))
```

```
Estado do vector apos aplicacao da porta H Statevector([ 0.70710678+0.j, -0.70710678+0.j, 0.      +0.j,
0.      +0.j],
             dims=(2, 2))
```



Aplicação da porta CNOT CX

```
#Aplicação da porta CNOT CX
qc = QuantumCircuit(2)
qc.x(0)
qc.h(0)
qc.cx(0, 1)
print(qc)
```



Calculo e Visualizacao do vector apos aplicacao da porta CNOT

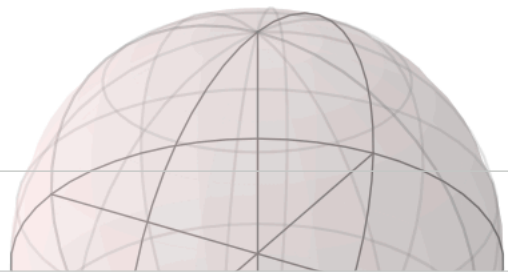
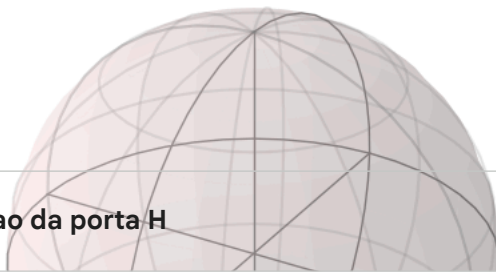
```
#Calculo e Visualizacao do vector apos aplicacao da porta CNOT
sv = Statevector(qc)
print('Estado do vector apos a aplicacao da porta CNOT', sv)

#Mostrar o vector na esfera de bloch
display(plot_bloch_multivector(sv))
```

```
Estado do vector apos a aplicacao da porta CNOT Statevector([ 0.70710678+0.j, 0.        +0.j, 0.        +0.j,  
                    -0.70710678+0.j],  
                    dims=(2, 2))
```

qubit 0
|0>

qubit 1
|0>



▼ Aplicacao da porta H

```
#Aplicacao da porta H  
qc = QuantumCircuit(2)  
qc.x(0)  
qc.h(0)  
qc.cx(0, 1)  
qc.h(0)  
print(qc)
```

|1>

|1>